JAVA MULTITHREADING & FILE HANDLING

30 Questions with Simple Answers

Beginner-friendly notes with concepts, differences, and short Java examples.

Prepared for study and exam revision

A. Multithreading in Java

1. What is a thread in Java?

Answer: A thread is a small unit of execution within a program. A Java program can use multiple threads to perform tasks concurrently.

2. What is multithreading?

Answer: Multithreading means running multiple threads within the same program. The threads may make progress concurrently and share the process's resources.

3. Why is multithreading used in Java applications?

Answer: It can keep applications responsive, perform background work, make better use of CPU resources, and allow independent tasks to progress at the same time.

4. What is the difference between a process and a thread?

Answer: A process is a running program with its own resources and memory space. Threads are execution paths within a process and generally share that process's memory and resources.

5. How can you create a thread in Java?

Answer: Common approaches are to extend the Thread class or implement Runnable and pass the task to a Thread. Example: Thread t = new Thread(() -> System.out.println("Hello")); t.start();

6. What is the difference between extending Thread and implementing Runnable?

Answer: Extending Thread creates a subclass of Thread, so the class cannot extend another class. Implementing Runnable separates the task from the thread and lets the class extend another class; it is often more flexible.

7. What is the purpose of the start() method?

Answer: start() asks the JVM to start a new thread. The new thread then executes its run() method.

8. What is the difference between start() and run()?

Answer: start() starts a separate thread of execution. Calling run() directly is an ordinary method call and runs on the current thread.

9. What is the life cycle of a thread?

Answer: Java thread states are NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. A thread moves between these states as it is started, scheduled, waits, obtains locks, and finishes.

10. What is the purpose of sleep()?

Answer: Thread.sleep(milliseconds) pauses the currently executing thread for at least the requested time, subject to scheduling. It can throw InterruptedException and does not release locks held by that thread.

11. What is the difference between sleep() and wait()?

Answer: sleep() pauses the current thread for a time and does not release monitors it holds. wait() is called on an object while owning that object's monitor; it releases that monitor and waits until notified, interrupted, or timed out.

12. What is thread synchronization?

Answer: Synchronization coordinates access to shared data so that threads do not interfere with each other and critical operations happen under controlled locking.

13. Why is the synchronized keyword used?

Answer: synchronized uses an object's monitor lock (or the class lock for a static synchronized method) to allow only one thread at a time to execute the protected section for that lock. It also provides memory-visibility guarantees.

14. What is a race condition?

Answer: A race condition occurs when the result depends on the unpredictable timing of threads accessing shared state. It can cause incorrect results, such as two withdrawals both using the same old balance.

15. What is the difference between join() and sleep()?

Answer: join() makes the current thread wait for another thread to finish (or for a specified time). sleep() pauses the current thread for a specified time without waiting for a particular thread to finish.

B. File Handling in Java

16. What is file handling in Java?

Answer: File handling means creating, reading, writing, updating, and managing files from a Java program.

17. Which package is commonly used for file handling in Java?

Answer: The java.io package contains many traditional file-handling classes. The java.nio.file package provides modern APIs such as Path and Files.

18. What is the purpose of the File class?

Answer: java.io.File represents a file or directory path and provides methods to check existence, obtain names, create directories, and inspect path properties. It does not itself read or write file contents.

19. How do you create a new file using Java?

Answer: Use File.createNewFile(). Example: File f = new File("notes.txt"); boolean created = f.createNewFile(); This can throw IOException, so handle it or declare it with throws.

20. How do you check whether a file exists?

Answer: Use exists(). Example: File f = new File("notes.txt"); if (f.exists()) { System.out.println("File exists"); }

21. How do you obtain the name and path of a file?

Answer: Use getName() for the final name, getPath() for the path as supplied, getAbsolutePath() for an absolute path, and getCanonicalPath() for a canonical path (which may throw IOException).

22. What is the difference between FileReader and FileWriter?

Answer: FileReader reads character data from a text file. FileWriter writes character data to a text file. Both use the default charset unless a charset-aware constructor or other API is chosen.

23. What is the difference between character streams and byte streams?

Answer: Character streams, such as FileReader and FileWriter, are designed for text. Byte streams, such as FileInputStream and FileOutputStream, read and write raw bytes and are suitable for binary files like images.

24. What is the purpose of FileInputStream?

Answer: FileInputStream reads raw bytes from a file. It is commonly used for binary data or as part of a byte-processing pipeline.

25. What is the purpose of FileOutputStream?

Answer: FileOutputStream writes raw bytes to a file. By default it creates or truncates the target file; a constructor with append=true can append data.

26. What is the difference between BufferedReader and FileReader?

Answer: FileReader reads characters from a file. BufferedReader wraps a Reader to buffer input efficiently and provides readLine() for convenient line-by-line reading.

27. How do you read a file line by line in Java?

Answer: Use BufferedReader with readLine(), preferably in try-with-resources: try (BufferedReader br = new BufferedReader(new FileReader("notes.txt"))) { String line; while ((line = br.readLine()) != null) { System.out.println(line); } }

28. How do you write data into a text file?

Answer: Use FileWriter or a buffered writer. Example: try (FileWriter writer = new FileWriter("notes.txt")) { writer.write("Hello Java"); } This overwrites the file by default; use an append option if needed.

29. Why should files be closed after reading or writing?

Answer: Closing files releases operating-system resources and helps flush buffered output. Try-with-resources automatically closes resources even if an exception occurs.

30. What exceptions can occur during file handling, and how can they be handled?

Answer: IOException is common; FileNotFoundException is a specific subclass. Other problems may include security or invalid-path errors. Handle exceptions with try-catch, use try-with-resources to close streams, and show a useful error message.